

SIMATS ENGINEERING

ASSESSMENT TOOL 2

COURSE CODE: ITA 0611

COURSE NAME: MACHINE LEARNING

COURSE OUTCOME COVERED

CO5: *Introduction – NumPy Arrays – NumPy Basics- Math – Random – Indexing – Filtering – Statistics – Aggregation – saving data – Data analysis with Pandas – DataFrame – Combining – Indexing – File I/O – Grouping – Filtering – Sorting – Plotting (BL 3)*

Assessment Tool Weightage: 20%

QUESTIONS:4

Total Marks:100

Annexure A

Pseudocode Writing Task (Algorithm Design)

Code of Conduct:

*I DHILIBAN M/192421166 (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.*

Signature of the student:

DHILIBAN M

Annexure A

Short Answer Test

1. Design a pseudocode algorithm to perform **data cleaning and transformation on time-series data**. The algorithm should handle missing timestamps, interpolate missing values, remove outliers, and resample data at fixed intervals. Include steps for indexing by time, filtering invalid entries, and ensuring chronological consistency while maintaining efficiency.
2. Write a detailed pseudocode algorithm to implement **feature selection** on a dataset. The algorithm should compute correlation between features, remove redundant or irrelevant attributes, and retain significant features. Include steps for normalization, threshold-based filtering, and validation to ensure improved model performance and reduced dimensionality.
3. Create a pseudocode algorithm to detect and handle **outliers in a dataset** using statistical methods such as Z-score or IQR. The algorithm should identify extreme values, decide whether to remove or cap them, and ensure that the resulting dataset maintains consistency and reliability for further analysis.
4. Write a pseudocode algorithm to implement **dimensionality reduction using Principal Component Analysis (PCA)**. The algorithm should include steps for standardizing the dataset, computing covariance matrix, extracting eigenvalues and eigenvectors, and selecting principal components. Ensure efficient matrix operations and validation of reduced dimensions.

RUBRICS FOR CALCULATION

Criteria	Weightage	Level 4 – Exemplary (5 Marks)	Level 3 – Proficient (4 Marks)	Level 2 – Developing (2–3 Marks)	Level 1 – Beginning (0–1 Mark)
Conceptual Understanding	25	Demonstrates clear and complete understanding of the concept	Good understanding with minor gaps	Basic understanding with some misconceptions	Poor or incorrect understanding
Accuracy of Answer	25	Completely correct answer with no errors	Mostly correct with minor errors	Partially correct with noticeable errors	Incorrect or irrelevant answer
Application of Knowledge	20	Correctly applies concepts to solve the question/problem	Applies concepts with minor mistakes	Limited or partially correct application	No or incorrect application
Completeness of Response	15	Covers all required parts of the question	Covers most parts with minor omissions	Covers only some parts	Incomplete or missing response
Clarity & Presentation	15	Clear, concise, and well-organized answer	Understandable with minor issues	Somewhat unclear or poorly structured	Difficult to understand

Q1. Pseudocode: Data Cleaning & Transformation of Time-Series Data

Conceptual Overview

Time-series cleaning has requirements ordinary tabular cleaning does not: the **index itself (time) can be invalid or missing**, values are **order-dependent** (chronological consistency matters), and outlier/missing-value handling must respect **temporal locality** (e.g., interpolate using neighboring timestamps, not the global mean). The algorithm below proceeds in six clearly separated stages so each constraint is verifiable independently.

Algorithm

ALGORITHM CleanTimeSeries(RawData, expected_freq, z_threshold)

INPUT: RawData -> list of (timestamp, value) records, possibly unsorted/irregular
expected_freq -> desired fixed interval (e.g., "1 hour", "1 day")
z_threshold -> Z-score cutoff for outlier detection (e.g., 3)
OUTPUT: CleanData -> chronologically consistent, evenly spaced, outlier-free series

STEP 1: PARSE & INDEX BY TIME

```
FOR each record r in RawData:
    CONVERT r.timestamp to standard datetime format
    IF conversion fails:
        LOG r as invalid_timestamp
        REMOVE r from RawData
SET RawData.index = timestamp // time becomes the primary index
SORT RawData BY timestamp ASCENDING // ensures chronological consistency
```

STEP 2: REMOVE DUPLICATE / INVALID TIMESTAMPS

```
duplicates <- FIND rows WHERE timestamp appears more than once
FOR each duplicate group:
    KEEP the record with the most complete/valid value (or average duplicates)
ASSERT RawData.index IS strictly increasing (no duplicate or out-of-order timestamps)
```

STEP 3: RESAMPLE TO FIXED INTERVALS

```
full_time_range <- GENERATE evenly spaced timestamps
    FROM RawData.index.min() TO RawData.index.max()
    AT interval = expected_freq
ResampledData <- REINDEX RawData ONTO full_time_range
    // this introduces NaN at newly created (previously missing) timestamps
MARK positions that were newly introduced as "missing"
```

STEP 4: HANDLE MISSING VALUES (INTERPOLATION)

```
FOR each missing value at time t in ResampledData:
    t_prev <- nearest valid timestamp BEFORE t
    t_next <- nearest valid timestamp AFTER t
    IF both t_prev and t_next exist:
        value[t] <- LINEAR_INTERPOLATE(value[t_prev], value[t_next], t)
    ELSE IF only one side exists (boundary case):
        value[t] <- FORWARD_FILL or BACKWARD_FILL from the available side
    ELSE:
        FLAG t for removal (entire series has no neighboring reference)
REMOVE any timestamps still flagged (isolated/unrecoverable gaps)
```

STEP 5: DETECT & REMOVE/CAP OUTLIERS

```

mean <- MEAN(ResampledData.value)
std <- STD(ResampledData.value)
FOR each value v at time t:
  z <- (v - mean) / std
  IF ABS(z) > z_threshold:
    // prefer capping over deletion to preserve time continuity
    v <- CLIP(v, mean - z_threshold*std, mean + z_threshold*std)
// Optionally use rolling window statistics instead of global mean/std,
// since time series often exhibit trend/seasonality (local outlier detection)

```

STEP 6: FINAL VALIDATION

```

ASSERT no missing values remain
ASSERT timestamps are strictly increasing and evenly spaced at expected_freq
ASSERT no value lies beyond [mean - z_threshold*std, mean + z_threshold*std]
RETURN CleanData

```

Efficiency & Consistency Notes

- **Indexing by time** (Step 1) allows all later operations (resampling, interpolation) to use $O(\log n)$ or $O(1)$ time-based lookups instead of linear scans.
- **Resampling before interpolation** (Steps 3→4) ensures interpolation fills exactly the fixed-interval grid, avoiding a second pass to re-align timestamps.
- **Local rolling statistics** are preferred over global mean/std for outlier detection in real deployments, since time series often have trend/seasonality that a global Z-score would misclassify as outliers — this is noted as a refinement in Step 5.
- Using **vectorized interpolation** (e.g., `pandas.Series.interpolate(method="time")`) rather than a per-point loop is the efficient real-world implementation of Step 4.

Q2. Pseudocode: Feature Selection Algorithm

Conceptual Overview

Feature selection must balance **removing redundancy** (features highly correlated with each other) against **removing irrelevance** (features weakly related to the target), while **normalization** ensures correlation/distance-based comparisons aren't distorted by differing feature scales. A validation step closes the loop by confirming the reduced feature set actually helps (or at least doesn't hurt) model performance.

Algorithm

```

ALGORITHM FeatureSelection(Data, target, corr_threshold, relevance_threshold)
INPUT: Data          -> DataFrame of features X and target y
      corr_threshold  -> max allowed correlation between two features (e.g., 0.85)
      relevance_threshold -> min required correlation with target (e.g., 0.05)
OUTPUT: SelectedFeatures -> reduced, non-redundant, relevant feature subset

```

STEP 1: PREPROCESSING / NORMALIZATION

```

FOR each numeric feature f in Data:
  Data[f] <- (Data[f] - MEAN(Data[f])) / STD(Data[f]) // z-score normalization

```

```

// Normalization ensures correlation and variance comparisons are scale-independent

STEP 2: REMOVE NEAR-ZERO VARIANCE / IRRELEVANT FEATURES
FOR each feature f in Data:
  IF VARIANCE(Data[f])  $\approx$  0:
    REMOVE f // constant features carry no information
  relevance_score <- ABS(CORRELATION(Data[f], target))
  IF relevance_score < relevance_threshold:
    REMOVE f // weakly related to target, likely irrelevant

STEP 3: COMPUTE FEATURE-FEATURE CORRELATION MATRIX
CorrMatrix <- COMPUTE pairwise correlation for all remaining features
// CorrMatrix[i][j] = correlation(feature_i, feature_j)

STEP 4: REMOVE REDUNDANT FEATURES (THRESHOLD-BASED FILTERING)
SORT remaining features BY relevance_score DESCENDING
SelectedFeatures <- empty list
FOR each feature f IN sorted order:
  is_redundant <- FALSE
  FOR each feature s IN SelectedFeatures:
    IF ABS(CorrMatrix[f][s]) > corr_threshold:
      is_redundant <- TRUE // f duplicates information already captured by s
      BREAK
  IF NOT is_redundant:
    ADD f TO SelectedFeatures
// Keeping the higher-relevance feature from each correlated pair/group
// ensures we retain the most target-informative representative

STEP 5: VALIDATION
Model_full <- TRAIN model USING all original features
Model_reduced <- TRAIN model USING SelectedFeatures
score_full <- EVALUATE Model_full ON validation set (e.g., accuracy / RMSE via cross-validation)
score_reduced <- EVALUATE Model_reduced ON validation set
IF score_reduced >= score_full - acceptable_margin:
  ACCEPT SelectedFeatures // dimensionality reduced without meaningful performance loss
ELSE:
  REVIEW relevance_threshold / corr_threshold and REPEAT from Step 2 (tune thresholds)

STEP 6: RETURN
RETURN SelectedFeatures

```

Efficiency & Correctness Notes

- Sorting by relevance before redundancy filtering (Step 4) guarantees that when two features are correlated, the **more predictive one is kept**, not an arbitrary one.
- Computing the correlation matrix once (Step 3) and reusing it (Step 4) avoids redundant $O(n^2)$ recomputation.
- The validation step (Step 5) is essential — correlation-based filtering is a heuristic; only downstream model performance confirms the selection actually improved (or preserved) predictive quality while reducing dimensionality.

Q3. Pseudocode: Outlier Detection & Handling (Z-score / IQR)

Conceptual Overview

Outlier handling requires **detection** (statistically defining what counts as extreme), a **decision policy** (remove vs. cap, based on context — e.g., whether the outlier is a data-entry error or a genuine rare event), and **post-treatment validation** to confirm the dataset remains statistically consistent.

Algorithm

ALGORITHM DetectAndHandleOutliers(Data, method, z_threshold, iqr_multiplier, policy)

INPUT: Data -> dataset with numeric column(s)

 method -> "zscore" or "iqr"

 z_threshold -> e.g., 3 (for Z-score method)

 iqr_multiplier -> e.g., 1.5 (for IQR method)

 policy -> "remove" or "cap"

OUTPUT: CleanData -> dataset with outliers handled, statistically consistent

STEP 1: SELECT DETECTION METHOD

IF method == "zscore":

 mean <- MEAN(Data.column)

 std <- STD(Data.column)

 FOR each value v in Data.column:

 z <- (v - mean) / std

 v.is_outlier <- (ABS(z) > z_threshold)

ELSE IF method == "iqr":

 Q1 <- 25th PERCENTILE(Data.column)

 Q3 <- 75th PERCENTILE(Data.column)

 IQR <- Q3 - Q1

 lower_bound <- Q1 - iqr_multiplier * IQR

 upper_bound <- Q3 + iqr_multiplier * IQR

 FOR each value v in Data.column:

 v.is_outlier <- (v < lower_bound) OR (v > upper_bound)

STEP 2: REVIEW FLAGGED OUTLIERS (CONTEXT CHECK)

outliers <- FILTER Data WHERE is_outlier == TRUE

// In practice: domain review distinguishes genuine extreme events (keep)

// from measurement/data-entry errors (remove/cap)

LOG outliers.count AND outliers.values for audit/debugging

STEP 3: APPLY HANDLING POLICY

IF policy == "remove":

 Data <- Data WHERE is_outlier == FALSE

ELSE IF policy == "cap": // Winsorization

 IF method == "zscore":

 lower_bound <- mean - z_threshold * std

 upper_bound <- mean + z_threshold * std

 FOR each value v WHERE is_outlier == TRUE:

 v <- CLIP(v, lower_bound, upper_bound)

STEP 4: RE-VALIDATE DATASET CONSISTENCY

new_mean <- MEAN(Data.column)

new_std <- STD(Data.column)

```

ASSERT MAX(Data.column) <= upper_bound
ASSERT MIN(Data.column) >= lower_bound
ASSERT Data.row_count > 0           // handling did not eliminate all data
COMPARE new_mean/new_std to original mean/std
  -> large shifts indicate outliers dominated the distribution;
      flag for further review rather than blind acceptance

```

```

STEP 5: RETURN
RETURN CleanData = Data

```

Design Notes

- **Z-score** assumes an approximately normal distribution and is sensitive to the outliers themselves (since they inflate mean/std); **IQR** is distribution-free and more robust for skewed data — the algorithm supports either, chosen based on data shape.
- **Capping (Winsorization)** is generally preferred over removal when preserving sample size/statistical power matters (e.g., small datasets); **removal** is preferred when the outlier is clearly a data-entry error unrelated to the true process being measured.
- Step 4's consistency check guards against over-aggressive cleaning (e.g., accidentally removing a large legitimate fraction of the data).

Q4. Pseudocode: Dimensionality Reduction via PCA

Conceptual Overview

PCA finds a new orthogonal basis (principal components) ordered by the variance they explain. The algorithm has four mathematical stages — **standardization, covariance computation, eigendecomposition, and projection** — followed by a validation step confirming the chosen number of components retains sufficient information.

Algorithm

```

ALGORITHM PCA(Data, variance_retain_threshold)
INPUT: Data           -> n x d matrix (n samples, d features)
      variance_retain_threshold -> e.g., 0.95 (retain 95% of total variance)
OUTPUT: ReducedData   -> n x k matrix (k < d, reduced dimensions)

```

STEP 1: STANDARDIZE THE DATASET

```

FOR each feature column f in Data:
  Data[f] <- (Data[f] - MEAN(Data[f])) / STD(Data[f])
// Standardization is essential: PCA is scale-sensitive; without it,
// high-magnitude features would dominate the variance/covariance structure

```

STEP 2: COMPUTE THE COVARIANCE MATRIX

```

CovMatrix <- (1 / (n - 1)) * TRANSPOSE(Data) MATMUL Data
// CovMatrix is a d x d symmetric matrix capturing pairwise feature covariances

```

STEP 3: EXTRACT EIGENVALUES AND EIGENVECTORS

```

(eigenvalues, eigenvectors) <- EIGENDECOMPOSITION(CovMatrix)
// Efficient computation: use Singular Value Decomposition (SVD) on the
// standardized data matrix directly instead of explicit eigendecomposition

```



```

// of the covariance matrix — SVD is more numerically stable and avoids
// ever explicitly forming the (potentially large) covariance matrix

STEP 4: SORT & SELECT PRINCIPAL COMPONENTS
SORT (eigenvalue, eigenvector) pairs BY eigenvalue DESCENDING
total_variance <- SUM(eigenvalues)
cumulative_variance <- 0
k <- 0
FOR each eigenvalue e IN sorted order:
  cumulative_variance <- cumulative_variance + e
  k <- k + 1
  IF (cumulative_variance / total_variance) >= variance_retain_threshold:
    BREAK
TopEigenvectors <- FIRST k eigenvectors (by sorted order)
// k is chosen adaptively to satisfy the variance-retention constraint,
// rather than being fixed arbitrarily

STEP 5: PROJECT DATA ONTO PRINCIPAL COMPONENTS
ProjectionMatrix <- STACK(TopEigenvectors) as columns // d x k matrix
ReducedData <- Data MATMUL ProjectionMatrix // n x k matrix

STEP 6: VALIDATE REDUCED DIMENSIONS
ASSERT ReducedData.shape == (n, k)
explained_variance_ratio <- cumulative_variance / total_variance
ASSERT explained_variance_ratio >= variance_retain_threshold
// Optional: reconstruct approximate original data and measure reconstruction error
Reconstructed <- ReducedData MATMUL TRANSPOSE(ProjectionMatrix)
reconstruction_error <- MEAN((Data - Reconstructed)^2)
LOG reconstruction_error // low error confirms k components adequately represent the data

STEP 7: RETURN
RETURN ReducedData, ProjectionMatrix, explained_variance_ratio

```

Efficiency & Validation Notes

- Computing PCA via **SVD directly on the standardized data matrix** (noted in Step 3) is the standard efficient implementation — it avoids explicitly building the $d \times d$ covariance matrix when d is large, and is numerically more stable than eigendecomposition of the covariance matrix.
- Selecting k via a **cumulative explained-variance threshold** (Step 4) is preferred over an arbitrary fixed k , since it directly ties dimensionality reduction to how much information is preserved.
- The **reconstruction error check** (Step 6) is a practical validation technique: it quantifies information loss and confirms the chosen k is adequate before using `ReducedData` in downstream models.